

AHMADU BELLO UNIVERSITY, ZARIA

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COEN838: ADVANCED SOFTWARE ENGINEERING

Project Examination, Question 4

OFFLINE-FIRST CRYPTOGRAPHIC QR IDENTITY WALLET

FOR RURAL PUBLIC SERVICES

SECURITY ENGINEERING REPORT

Document QRW-SEC-03 · Version 1.0

BY

ALIYU SARKI ILIYASU

P25EGCP8040

November 2026

DOCUMENT CONTROL

Document details

Item	Detail
Document	Security Engineering Report (QRW-SEC-03)
Project	Offline-First Cryptographic QR Identity Wallet for Rural Public Services
Course	COEN838 Advanced Software Engineering, Ahmadu Bello University, Zaria
Author	Aliyu Sarki Iliyasu (P25EGCP8040)
Version / date	1.0 · November 2026
Source code	https://github.com/sarki94/qr-identity-wallet
Related documents	QRW-SRS-01 Requirements · QRW-SDD-02 Design · QRW-TST-04 Test Plan & Report

Revision history

Version	Date	Change
1.0	4 Nov 2026	First complete security engineering report, including OWASP ZAP scan

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Purpose....1
1.2 Scope....1
1.3 Method....1
1.4 References....1
2. ASSETS AND SECURITY OBJECTIVES....3
2.1 Assets....3
2.2 Security Objectives....3
3. TRUST BOUNDARIES AND DATA FLOWS....4
4. STRIDE THREAT MODEL....5
5. SECURE-DESIGN PRINCIPLES....7
5.1 Defence in Depth at the Distribution Point....7
6. AUTHENTICATION AND AUTHORISATION SCHEME....9
6.1 Holders (Farmers), Offline....9
6.2 Staff, Online....9
6.3 Service to Service....9
6.4 Authorisation....10
6.5 Cryptographic Choices....10
7. SECURITY TESTING AND EVIDENCE....11
7.1 Static Analysis....11
7.2 Dependency Scanning....11
7.3 Dynamic Scanning with OWASP ZAP....12
7.4 Security Test Cases....13
8. RESIDUAL RISKS AND RECOMMENDATIONS....15

1. INTRODUCTION

1.1 Purpose

This report sets out how security was engineered into the Offline-First Cryptographic QR Identity Wallet. It identifies the assets worth protecting and the trust boundaries they cross, models the threats with STRIDE, shows how the secure-design principles of least privilege, defence in depth and failing securely were applied, describes the authentication and authorisation scheme, and presents the evidence from static analysis, dependency scanning, dynamic scanning and security test cases. Remaining risks are carried to the risk register in QRW-PRM-05.

1.2 Scope

The report covers the cryptographic core library, the five services (Auth, Issuer, Trust and Revocation, Audit and Sync, and the mock NIMC) and the web application with its Wallet, Verifier, Issuer Portal and Audit screens. Physical security of distribution points, the security of NIMC itself and the device operating systems are outside its scope and are recorded as assumptions.

1.3 Method

Threats were identified with Microsoft's STRIDE model, as taught in the course, applied to each trust boundary of the data-flow diagram in Section 3. Each threat is stated concretely for this system, linked to the security property it violates, and given a mitigation that names the test cases verifying it. The test identifiers match the Requirements Traceability Matrix (QRW-SRS-01, Appendix A) and the Test Plan and Report (QRW-TST-04).

1.4 References

Josefsson, S., & Liusvaara, I. (2017). *Edwards-curve digital signature algorithm (EdDSA)* (RFC 8032). Internet Engineering Task Force. https://doi.org/10.17487/RFC8032

Moriarty, K., Kaliski, B., & Rusch, A. (2017). *PKCS #5: Password-based cryptography specification version 2.1* (RFC 8018). Internet Engineering Task Force. https://doi.org/10.17487/RFC8018

Nigeria Data Protection Commission. (2023). *Nigeria Data Protection Act, 2023*. Federal Republic of Nigeria.

OWASP Foundation. (2021). *OWASP top 10: 2021*. https://owasp.org/Top10/

Percival, C., & Josefsson, S. (2016). *The scrypt password-based key derivation function* (RFC 7914). Internet Engineering Task Force. https://doi.org/10.17487/RFC7914

Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, *63*(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Shostack, A. (2014). *Threat modeling: Designing for security*. Wiley.

2. ASSETS AND SECURITY OBJECTIVES

2.1 Assets

The assets are listed in Table 2.1 in order of value. The ranking drives the design: the most valuable asset, the issuer signing key, is held by only one component.

Table 2.1: Assets ranked by value

Rank	Asset	Why it matters	Where it lives
1	Issuer signing key (Ed25519)	Whoever holds it can create identities that every verifier accepts	Issuer service only; encrypted at rest
2	Holder device keys	Prove that the person presenting is the person enrolled	Each wallet profile, sealed under its PIN
3	Revocation list	A forged or stale list lets revoked credentials pass	Issuer (signs), Trust (publishes), verifier caches
4	Audit records	Evidence of who was verified, where, and with what result	Verifier queue, then the Audit service
5	Personal data (NIN, name, ward, photo)	Protected under the Nigeria Data Protection Act 2023	Issuer database (NIN hashed), credential (NIN masked)
6	Staff accounts and tokens	Grant the power to issue, revoke, verify or read audit logs	Auth service; tokens on staff devices

2.2 Security Objectives

The security requirements of QRW-SRS-01 are restated in Table 2.2 with the property each protects.

Table 2.2: Security requirements and properties

ID	Requirement	Property
NFR-S1	Ed25519 signing; issuer private key only in the Issuer, encrypted at rest (HSM in production)	Integrity, authenticity
NFR-S2	Holder private keys sealed with a PIN-derived key (PBKDF2 and AES-GCM)	Confidentiality, authentication
NFR-S3	TLS on all traffic (prototype: HTTPS between phones and the app)	Confidentiality, integrity
NFR-S4	Data minimisation: masked NIN only; full NIN never stored	Confidentiality, privacy
NFR-S5	Fail securely: any error, unknown key or bad input is a rejection	Integrity, availability
NFR-S6	Audit records tamper-evident (hash chain)	Non-repudiation, integrity
NFR-S7	Login rate-limited; account locks after five failed logins	Authentication, availability
NFR-S8	Nigeria Data Protection Act 2023 principles	Privacy, lawfulness

3. TRUST BOUNDARIES AND DATA FLOWS

Figure 3.1 shows the data flows and the six trust boundaries (B1 to B6) at which data passes between parties that trust each other differently. Each boundary is a place where an attacker can inject, observe or alter data, so the STRIDE analysis in Section 4 is applied boundary by boundary.

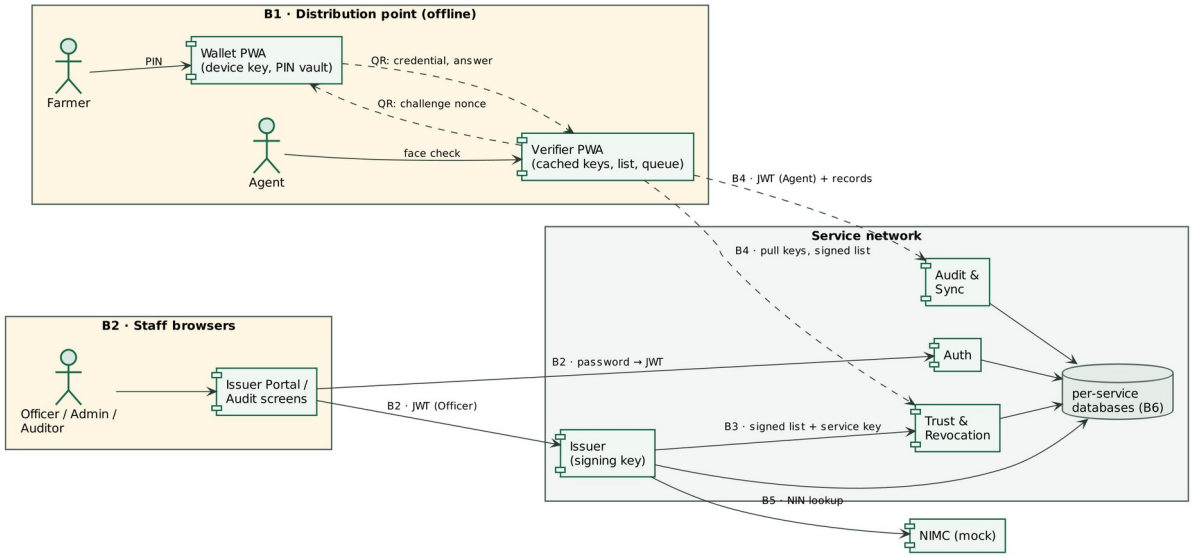


Figure 3.1: Data-flow diagram with trust boundaries

Table 3.1: Trust boundaries

ID	Between	Crossing protected by
B1	Farmer's wallet ↔ agent's verifier (offline, QR)	Issuer signature, holder challenge–response, face check
B2	Staff browsers ↔ services	Staff login, EdDSA JWT, role checks, HTTPS (phones)
B3	Issuer → Trust (internal publish)	Service key; Trust re-verifies the list signature
B4	Verifier ↔ Trust / Audit (async sync)	Verifier checks list signature itself; Audit takes identity from the token
B5	Issuer → Mock NIMC	Internal network only (production: NIMC-issued credentials, mutual TLS)
B6	Each service ↔ its own database	Database per service; no service reads another's data

4. STRIDE THREAT MODEL

The threat model uses the four-column STRIDE format given in the course: threat category, security property violated, definition and mitigation. The definition states the concrete threat in this system, numbered by category (S1, T1 and so on); the mitigation names the controls and, in italics, the test cases that verify them.

Nineteen threats were identified across the six categories (Table 4.1). Every threat has at least one implemented mitigation verified by at least one test; for R2 a residual part remains, noted in the table and in Section 8.

Table 4.1: STRIDE threat model

Threat category	Security property violated	Definition (threat in this system)	Mitigation (and verifying tests)
Spoofing	Authentication	S1. An impostor presents another farmer's credential: a screenshot, photocopy or borrowed phone (the proxy collection observed in Kura, F1).	L2: offline challenge–response. The wallet signs a fresh 128-bit nonce + credential ID with a device key that never leaves the phone. PIN unlocks the key; the agent's face check comes last. *TC-CORE-031, -032, -033, TC-WEB-007, TC-SYS-002 (replay rejected), TC-SYS-003 step 10.*
Spoofing	Authentication	S2. An attacker forges a staff token or guesses a password to act as an officer or admin.	Passwords hashed with scrypt; EdDSA-signed JWTs with a fixed algorithm list (rejects alg: none and HS256 key confusion); 5-attempt lockout; rate limit on login. *TC-SEC-004, -005, TC-SVC-012, -013, -016.*
Spoofing	Authentication	S3. A fake "issuer" or fake revocation list is fed to a verifier.	Verifiers trust only keys from the Trust service and verify the list signature themselves before caching it. *TC-WEB-008, TC-CORE-042, TC-SVC-025.*
Tampering	Integrity	T1. Someone edits a credential (for example the name) to collect someone else's fertiliser.	Ed25519 signature over the exact payload bytes; the verifier checks the signature before reading any field. *TC-CORE-013, TC-CORE-014 (every single-bit flip rejected), TC-WEB-003, TC-SYS-002.*
Tampering	Integrity	T2. Someone removes an entry from the revocation list, or serves an older list (rollback).	List is signed by the Issuer; Trust verifies and refuses rollback; the verifier also refuses an older version. *TC-CORE-041, TC-SVC-025, TC-WEB-008.*
Tampering	Integrity	T3. An insider edits or deletes audit records in the database.	SHA-256 hash chain: each record stores the previous record's hash; edits, deletions and reordering break the chain. *TC-CORE-051, -052, -053, TC-SVC-031.*
Repudiation	Non-repudiation	R1. An agent denies a verification, or files records in another agent's name.	Agent and point are taken from the signed token, never the request body; every attempt is recorded and hash-chained. *TC-SEC-006, TC-SVC-030, TC-SVC-031.*
Repudiation	Non-repudiation	R2. A farmer denies having presented the credential.	The wallet's answer is a digital signature over the nonce and credential ID. (Residual: the prototype records the result, not the signed answer; storing it is a planned perfective change.) *TC-CORE-030.*
Information disclosure	Confidentiality	I1. The QR (seen by every agent) exposes personal data.	Data minimisation: masked NIN (123•••••901), no BVN, phone number or date of birth. *TC-CORE-018, TC-SVC-020.*

Threat category	Security property violated	Definition (threat in this system)	Mitigation (and verifying tests)
Information disclosure	Confidentiality	I2. A stolen Issuer database reveals NINs.	Full NIN never stored: HMAC-SHA256 keyed hash (secret pepper) plus masked copy (D13). *TC-SVC-020.*
Information disclosure	Confidentiality	I3. A stolen or shared phone reveals another family member's key.	Each profile's key is sealed with its own PIN (PBKDF2 210k + AES-GCM); 5 wrong PINs lock the profile; the credential can be revoked. *TC-WEB-014.*
Information disclosure	Confidentiality	I4. Error messages reveal internals; responses are cached or framed by other sites.	Generic error handler (no stack traces); nosniff, X-Frame-Options: DENY, no-store, strict CSP on every response; CORS limited to the app's origin. *TC-SEC-008, -009, -010.*
Denial of service	Availability	D1. No network at the distribution point (the exam's problem statement).	Offline-first: verification needs no server; records queue locally. *TC-SYS-002, TC-SYS-003 steps 6–11.*
Denial of service	Availability	D2. Login flooding or password guessing.	Rate limit (20/min per client) and lockout. *TC-SVC-016, TC-SVC-012.*
Denial of service	Availability	D3. Oversized uploads or a "decompression bomb" QR exhaust memory.	8 KB inflate cap on QR payloads; 2,000-record batch cap; field length limits. *TC-CORE-024, TC-SEC-007.*
Denial of service	Availability	D4. The Trust service is down.	Verifiers keep their cached list; the Issuer retries publishing. *TC-SVC-026.*
Elevation of privilege	Authorization	E1. An agent calls officer or admin functions.	Role check on every endpoint (least privilege). *TC-SVC-014.*
Elevation of privilege	Authorization	E2. A compromised Trust or Audit service gains the power to create identities or tokens.	Signing key only in the Issuer (D11); services hold only public keys to check tokens (D12); Trust can publish but not sign. *TC-SVC-025.*
Elevation of privilege	Authorization	E3. Injection (SQL, template, script) through inputs.	Parameterised queries (SQLAlchemy), strict Pydantic validation (e.g. NIN must be 11 digits). *TC-SEC-001, -002, -003.*

The mitigations correspond to those given in the course for each category: digital signatures and HMACs for tampering, signed audit logs for repudiation, encryption and access control for information disclosure, and rate limiting and quotas for denial of service. Multi-factor authentication for staff and mutual TLS between services were considered and deferred; Section 8 explains why and what replaces them in the prototype.

5. SECURE-DESIGN PRINCIPLES

Table 5.1 shows where each principle from Saltzer and Schroeder (1975), as taught in the course, is applied. The first three are the ones named in the examination rubric.

Table 5.1: Secure-design principles applied

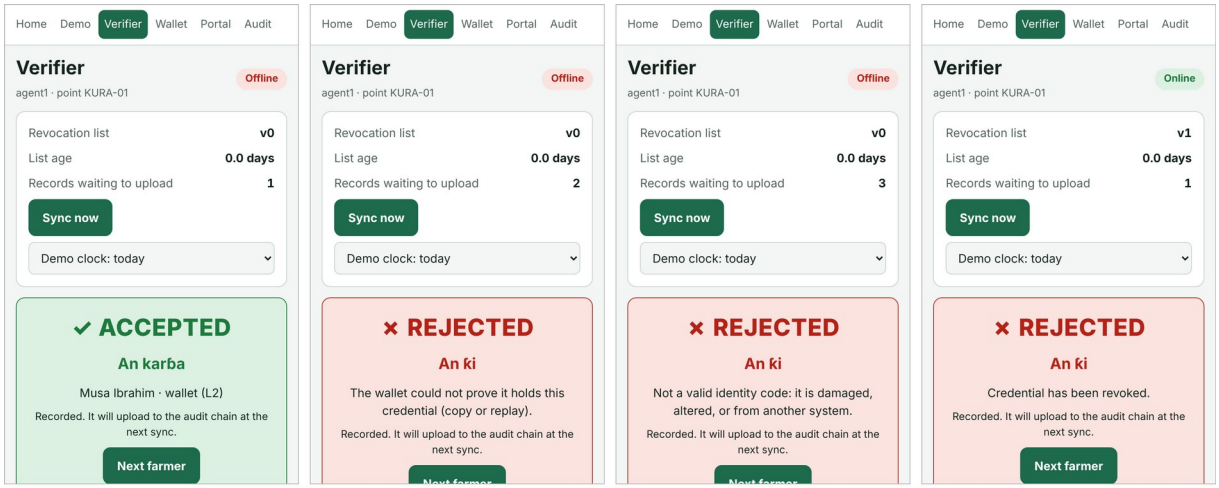
Principle	Where it is applied
Least privilege	Only the Issuer reads the signing key. Services verify tokens with Auth's public key and cannot mint them. Each role reaches only its own endpoints. Verifiers can check credentials but never create them.
Defence in depth	A credential must pass four independent layers: issuer signature, holder challenge, revocation/expiry, and the human face check. The revocation list is verified by Trust *and* again by each verifier.
Fail securely	Every unknown, malformed or expired input is rejected, never accepted or crashed on (property-tested over random input, TC-CORE-019, -023). Unreadable phone cache forces a fresh sync. Server errors return a generic message.
Minimise attack surface	No message broker or extra infrastructure; the web app's router dependency was removed when audited (see evidence/dependency-audit.md); credentials carry only what a distribution point needs.
Separation of duties	Officers issue and revoke; agents verify; auditors read logs; admins manage accounts. No role can both issue and audit.
Secure by default	Demo accounts can be switched off (QRW_SEED_DEMO=0); keys are generated encrypted; CORS defaults to the app's own origin.
Economy of mechanism	One signature scheme (Ed25519) and one hash (SHA-256) everywhere; the verifier logic is a small pure function, tested in both Python and the browser against the same vectors.

5.1 Defence in Depth at the Distribution Point

The clearest example of defence in depth is the check made on each farmer. Four layers must all pass, and each catches an attack the others cannot:

1. **Issuer signature.** Catches forged and edited credentials (T1).
2. **Holder challenge–response (L2).** Catches screenshots, photocopies and replayed answers (S1).
3. **Revocation and expiry.** Catches credentials that were valid but have been withdrawn (T2).
4. **Human face check.** Catches a genuine phone held by the wrong person, and is the only holder check for paper wallets (L1).

Figure 5.1 shows the verifier's verdicts for a genuine holder and for three of these attacks, captured from the automated browser test TC-SYS-002 with the phone offline.



(a) Genuine holder: accepted (b) Replayed answer: rejected (c) Edited credential: rejected (d) Revoked credential: rejected

Figure 5.1: Verifier verdicts: genuine, replayed, edited and revoked credentials

6. AUTHENTICATION AND AUTHORISATION SCHEME

6.1 Holders (Farmers), Offline

A farmer is authenticated at the distribution point without any network, by three factors: something they have (the device key in their wallet, proved by challenge–response), something they know (the six-digit PIN that unseals the key) and something they are (their face, checked by the agent against the photograph and name). The protocol is:

1. The wallet shows its credential QR; the verifier checks the issuer signature, revocation and expiry.
2. The verifier shows a challenge QR containing a fresh 128-bit random nonce and the credential ID.
3. The farmer enters the PIN; the wallet unseals its key and signs the message ["QRW1-CHALLENGE", nonce, credential ID].
4. The verifier checks the answer against the holder public key inside the signed credential. A nonce is accepted once only, so a recorded answer cannot be replayed.
5. The agent compares the face with the photograph and records the verdict.

Paper wallets (L1) cannot answer a challenge, so the face check is their only holder check. Every acceptance records its assurance level so that auditors can see how much L1 is used.

6.2 Staff, Online

Officers, agents, auditors and administrators log in to the Auth service with a username and password. Passwords are hashed with scrypt; five failed attempts lock the account (HTTP 423) and the login endpoint is rate-limited to 20 requests per minute per client. Auth then issues a JSON Web Token signed with EdDSA (Ed25519) carrying the claims in QRW-SDD-02 Table 6.2. Staff tokens last eight hours; agent tokens last seven days so that a verifier can keep working offline. Every other service verifies tokens with Auth's public key and a fixed algorithm list, which defeats the "alg: none" and HS256 key-confusion attacks.

6.3 Service to Service

The Issuer publishes revocation lists to Trust with a shared service key. Trust then re-verifies the Issuer's signature on the list, so a leaked service key alone cannot inject a

forged list: at worst it could re-send a list the Issuer had already signed, and the anti-rollback check refuses older versions. In production this channel would use mutual TLS.

6.4 Authorisation

Authorisation is role-based. Each endpoint declares the roles allowed, as shown in Table 6.1; separation of duties means no role can both issue credentials and audit verifications.

Table 6.1: Role permissions

Operation	Officer	Agent	Admin	Auditor
Enrol farmer, issue, revoke	Yes			
Republish revocation list	Yes		Yes	
View farmer's credentials	Yes		Yes	
Upload verification records		Yes		
Read records, check chain, anomalies			Yes	Yes
Create / disable / unlock staff			Yes	
Public keys and revocation list	open	open	open	open

6.5 Cryptographic Choices

The algorithms and parameters are listed in Table 6.2. Only standard, widely reviewed primitives are used, from maintained libraries (Python cryptography and PyJWT; @noble/ed25519 and the browser's Web Crypto API).

Table 6.2: Cryptographic algorithms and parameters

Algorithm	Used for	Parameters	Standard
Ed25519	Credentials, revocation lists, challenge answers, staff tokens	Strict verification in the browser (no ZIP-215 leniency)	RFC 8032
SHA-256	Audit hash chain; issuer key IDs	256-bit output	FIPS 180-4
HMAC-SHA256	Keyed hash of the NIN	256-bit secret pepper	RFC 2104
scrypt	Staff passwords	N = 2^14, r = 8, p = 1; 128-bit salt per user	RFC 7914
PBKDF2-SHA256 + AES-256-GCM	Wallet PIN vault	210,000 iterations; 128-bit salt; 96-bit IV	RFC 8018; NIST SP 800-38D
PKCS #8 encryption	Issuer signing key at rest	Best available encryption under a password	RFC 5958
CSPRNG	Nonces, salts, credential and record IDs	128-bit nonces	Web Crypto; Python secrets

7. SECURITY TESTING AND EVIDENCE

Security was tested with five complementary techniques, summarised in Table 7.1. Static analysis and dependency scanning run on every push in GitHub Actions, so a regression fails the build.

Table 7.1: Security testing performed

Technique	Tool	Result
Static analysis (SAST)	Bandit, Flake8	One low finding (B105 on the OAuth2 token type "bearer") triaged as a false positive; zero issues after triage
Dependency scanning (SCA)	pip-audit, npm audit	Python clean. Web: react-router-dom advisories found; the dependency was removed, leaving zero runtime vulnerabilities
Dynamic scanning (DAST)	OWASP ZAP 2.16.1	Before: 0 High, 1 Medium, 1 Low. One Medium fixed test-first. After: 0 High, 0 Medium, 1 Low accepted
Security test cases	pytest (black-box)	24 cases, TC-SEC-001 to TC-SEC-012: all pass
Property-based fuzzing	Hypothesis	Every single-bit flip of a credential is rejected; arbitrary QR text never crashes the decoder

7.1 Static Analysis

Bandit scanned the services and the core library. Its single finding, B105 (possible hard-coded password) on the string "bearer", is the OAuth2 token type defined in RFC 6750 and not a secret; it was marked as a reviewed false positive in the code with a comment explaining why. Bandit also flagged the use of subprocess with shell=True in a developer tool; that was fixed rather than suppressed. Before and after reports are kept in docs/evidence.

7.2 Dependency Scanning

pip-audit found no known vulnerabilities in the Python dependencies. npm audit found open-redirect and cross-site scripting advisories in react-router-dom, which is shipped to phones. Upgrading removed the high-severity advisories but left two moderate ones fixable only in a breaking major version, so the dependency was replaced by a 50-line internal router that navigates only to six fixed paths. The runtime audit now reports zero vulnerabilities. Advisories remaining in development-only tools (Vite, Vitest) do not reach users and are scheduled as adaptive maintenance.

7.3 Dynamic Scanning with OWASP ZAP

OWASP ZAP, the dynamic scanner named in the course, was run against all five running services. Each service's OpenAPI description was imported so that ZAP knew every endpoint, a valid token for a suitable role was attached to every request, and the full default active-scan policy was applied (SQL injection, cross-site scripting, path traversal, command injection, server-side request forgery and others); each scan sent about 6,860 requests. The scan script is in the repository (tools/zap_scan.py) so that it can be repeated.

The first scan (Figure 7.1) found no High-risk issues and one Medium: the interactive API documentation pages (/docs) had no Content-Security-Policy header. The API endpoints themselves already sent a strict policy, but the documentation pages had been exempted so that Swagger UI could load.

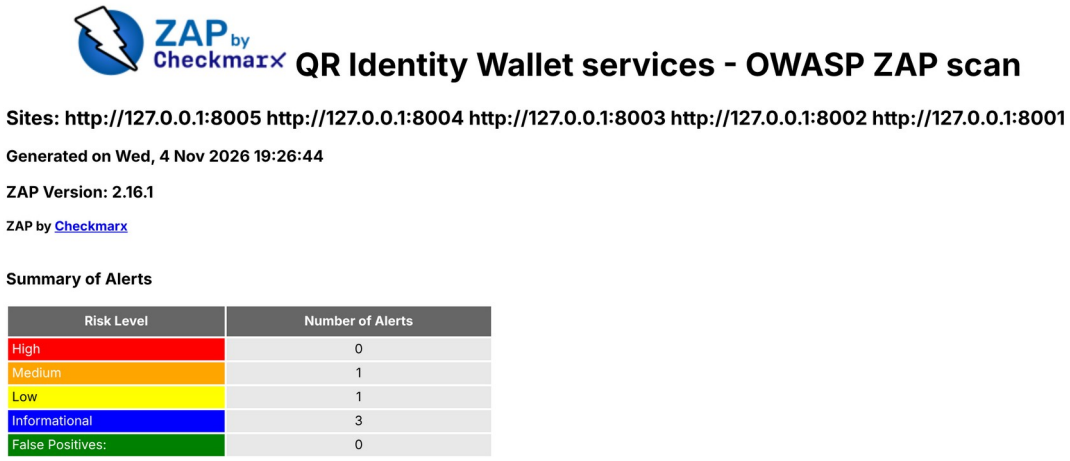


Figure 7.1: OWASP ZAP summary before the fix

The finding was fixed test-first. Two failing tests were written (TC-SEC-011 requires a strict policy on the documentation pages; TC-SEC-012 requires that the pages can be switched off), then the fix: the documentation pages now receive their own policy that allows scripts only from the service itself, the pinned CDN, or the page's own inline script identified by its SHA-256 hash, with no 'unsafe-inline' or 'unsafe-eval' and no framing; and setting QRW_API_DOCS=0 removes the pages and the OpenAPI description in production. A first version of the fix still allowed inline styles, and the re-scan raised a new Medium (rule 10055); inline styles were then removed after checking in a browser that Swagger UI still works without them. This loop of scan, test, fix and re-scan is the intended use of a DAST tool.

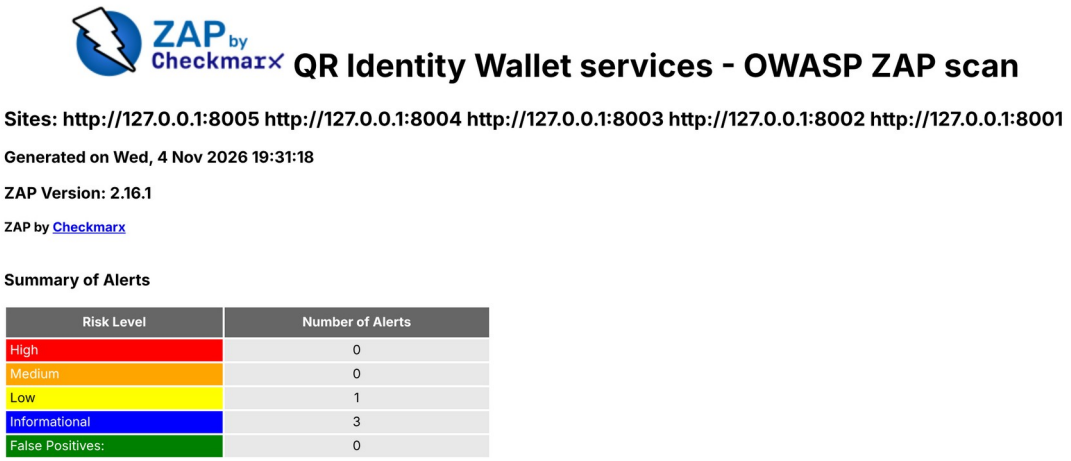


Figure 7.2: OWASP ZAP summary after the fix

The remaining alerts, all triaged, are listed in Table 7.2. During the scan the login rate limiter returned HTTP 429 to ZAP's fuzzing of the login endpoint, which is the intended behaviour (NFR-S7).

Table 7.2: Remaining ZAP alerts and triage

Risk	Alert	Triage
Low	Cross-domain JavaScript source file inclusion (10017), /docs pages	Accepted: developer documentation page; CDN pinned in the policy; page switched off in production (TC-SEC-012)
Info	Authentication request identified (10111)	Expected: the login endpoint
Info	Modern web application (10109)	Informational only
Info	User agent fuzzer (10104)	No behaviour change by user agent; no action

7.4 Security Test Cases

The black-box security tests in Table 7.3 run against the services on every push. Several are parameterised over a list of attack strings, giving 24 executed cases.

Table 7.3: Security test cases

ID	What it checks	Result
TC-SEC-001	SQL, template and script injection strings in the login form are rejected cleanly (6 payloads)	Pass
TC-SEC-002	Injection strings and malformed NINs are refused by input validation (8 payloads)	Pass
TC-SEC-003	Injection in audit query filters returns no rows and no error	Pass
TC-SEC-004	Forged tokens are rejected: unsigned (alg none) and HS256 key-confusion attacks	Pass
TC-SEC-005	Expired tokens are rejected	Pass
TC-SEC-006	An agent cannot file records in another agent's name	Pass
TC-SEC-007	Oversized photos, batches and fields are refused	Pass
TC-SEC-008	Security headers present on every response	Pass
TC-SEC-009	CORS allows only the web application's origin	Pass
TC-SEC-010	Server errors never reveal stack traces or internals	Pass
TC-SEC-011	API documentation pages carry a strict, hash-based CSP (ZAP finding)	Pass
TC-SEC-012	API documentation can be switched off for production	Pass

Security behaviour is also covered by functional tests listed against each threat in Table 4.1, for example the replay test in TC-SYS-002 and the single-bit-flip property test TC-CORE-014.

8. RESIDUAL RISKS AND RECOMMENDATIONS

No system is free of risk. Table 8.1 lists what remains after the mitigations, with the control that limits each and the recommended next step; the first five are carried to the risk register (QRW-PRM-05).

Table 8.1: Residual risks

Residual risk	Current control	Recommendation
Revocation lag at offline points: a revoked credential can pass until the verifier next synchronises	Seven-day warning and 30-day refusal on list age; credential expiry	Delta lists that are small enough to sync over SMS or USSD
Paper wallets (L1) rely on a human face check only	Level recorded on every acceptance; auditors can monitor L1 use	Prefer L2 wherever a smartphone is available; train agents
Phone cache tampering by someone with an unlocked agent phone	List-age rules; device screen lock	Cache the signed list and re-verify it on every load
Offline PIN guessing on a seized phone	PBKDF2 slows guessing; five-attempt lock; revoke on reported loss	Use the phone's hardware-backed key store where available
Signing key compromise	Key only in the Issuer, encrypted at rest	Hardware security module; key rotation through key IDs
Repudiation by a farmer (R2)	Signed answer is verified but not stored	Store the signed answer in the audit record
Staff accounts protected by password only	scrypt, lockout, rate limit, short token lifetime	Add multi-factor authentication (deferred for time)
Internal traffic not encrypted in the prototype	Services on one host behind a single HTTPS origin	Mutual TLS between services in production

Two controls named in the course were consciously deferred. Multi-factor authentication for staff was cut from the schedule recovery plan because the staff threat is already bounded by lockout, rate limiting and short-lived tokens, while the offline holder threat, which has no such bounds, received the effort instead. Mutual TLS was deferred because every service runs on one host in the prototype; it is the first change needed for a multi-host deployment.